

UD8: Automatización. JOBS

Índice

Introducción.....	2
Permisos.....	2
CREATE_JOB.....	3
STORED_PROCEDURE.....	4
External script.....	6
Run_job.....	7
Monitorizar el status y log de un job.....	7

Introducción

Un job es una combinación de un programa en PL/SQL y una planificación junto con el número de argumentos adicionales requeridos por el PL/SQL. Para trabajar con los jobs existe un proceso planificador en background cuyo nombre comienza por cjq y un conjunto de procesos hijos que son arrancados por el planificador y que ejecutan los jobs. La información sobre los jobs se guarda en la tabla **DBA|ALL|USER_SCHEDULER_JOBS**

Permisos

Task	Privilege Needed
Create a job	CREATE JOB or CREATE ANY JOB
Alter a job	ALTER or CREATE ANY JOB or be the owner
Run a job	ALTER or CREATE ANY JOB or be the owner
Copy a job	ALTER or CREATE ANY JOB or be the owner
Drop a job	ALTER or CREATE ANY JOB or be the owner
Stop a job	ALTER or CREATE ANY JOB or be the owner
Disable a job	ALTER or CREATE ANY JOB or be the owner
Enable a job	ALTER or CREATE ANY JOB or be the owner

Para poder trabajar con jobs precisamos los privilegios que aparecen en la tabla izquierda

Para trabajar con jobs hay que usar el paquete **DBMS_SCHEDULER** cuyos procedimientos son:

CREATE_JOB

ALTER_JOB

RUN_JOB (nombre)

COPY_JOB (nombre1, nombre2)

DROP_JOB (nombre)

STOP_JOB (nombre)

DISABLE (nombre,TRUE|FALSE)

ENABLE (nombre)

Hay que tener en cuenta que los jobs se ejecutan con los privilegios del esquema en el que se crean y por defecto están deshabilitados

CREATE_JOB

```
DBMS_SCHEDULER.CREATE_JOB (  
    job_name IN VARCHAR2, -- nombre del job  
    job_type IN VARCHAR2, -- PLSQL_BLOCK, STORED_PROCEDURE, EXTERNAL_SCRIPT  
    job_action IN VARCHAR2, -- nombre del programa PL/SQL o sentencias a realizar  
    number_of_arguments IN PLS_INTEGER DEFAULT 0, -- argumentos del job  
    start_date IN TIMESTAMP WITH TIME ZONE DEFAULT NULL, -- cuando comienza.  
                                Null para comience cuando se habilite  
    repeat_interval IN VARCHAR2 DEFAULT NULL, -- intervalo repetición. Si nulo sólo  
                                se hace unha vez  
    end_date IN TIMESTAMP WITH TIME ZONE DEFAULT NULL, -- data fin. Si null lo  
                                hará infinitamente  
    enabled IN BOOLEAN DEFAULT FALSE, -- habilitado  
    auto_drop IN BOOLEAN DEFAULT TRUE, -- borra el job una vez deshabilitado  
);
```

El intervalo de tempo se define por dos partes:

FREQ : Frecuencia

MINUTELY

HOURLY

DAILY

MONTHLY

YEARLY

INTERVAL : Intevalo de repetición

BYDAY : CADA DÍA DA SEMANA (MON,TUE,...)

BYMONTHDAY : CADA DÍA DO MES (-A,...+A)

BYDATE : CADA DATA (DDMM)

BYHOUR : CADA HORA (00,...24)

INTERVAL=N CADA N minutos, días, horas... dependiendo de la frecuencia

Si el JOB_TYPE es un **PLSQL_BLOCK** ponemos una sentencia SQL o varias rodeadas de **BEGIN... END**;

Para poner los atributos podemos hacerlo en el create o con el procedimiento set_attribute, por ejemplo

BEGIN

DBMS_SCHEDULER.SET_ATTRIBUTE('job_operations','max_runs',3);

END;

/

Para poner un atributo a null sin embargo no podemos usar set_attribute sinó set_attribute_null

Ejemplo:

BEGIN

DBMS_SCHEDULER.SET_ATTRIBUTE_NULL('job_operations','max_runs');

END;

STORED_PROCEDURE

Podemos hacer un job que ejecute un procedimiento almacenado

Ejemplo:

```
BEGIN
DBMS_SCHEDULER.CREATE_JOB (
    job_name => 'update_sales',
    job_type => 'STORED_PROCEDURE',
    job_action => 'OPS.SALES_PKG.UPDATE_SALES_SUMMARY',
    start_date => sysdate,
    repeat_interval => 'FREQ=DAILY;INTERVAL=2', /* cada dos días */
    end_date => sysdate+30
);
END;
/
```

Si el JOB_TYPE es un STORED_PROCEDURE ponemos el nombre del procedimiento (sin paréntesis)

END_DATE. Es la fecha en la que el job se ejecutará por última vez. (El estado (STATE) del job se asigna a COMPLETED, y se deshabilita).

Dado que nombre del procedimiento va sin argumentos tenemos que ponerlos a parte con set_job_argument_value que debemos usar con un nuevo objeto llamado programa. Cuando creamos jobs que admiten argumentos (ya que el procedimiento al que llama los admite) tenemos que ponerlos en modo disabled y cambiarlos a enabled después de establecer los argumentos, si no fallará.

Ejemplo

-- creamos el procedimiento

```
CREATE OR REPLACE PROCEDURE MI_PROCEDURE (P_VAR1 VARCHAR2, P_VAR_NUMBER
NUMBER)
AS
    V_RESULT_MSG VARCHAR2(300);
BEGIN
    DBMS_OUTPUT.PUT_LINE(TO_CHAR(SYSDATE,'DD/MM/YYYY HH24:MI'));
    V_RESULT_MSG := P_VAR1||' '||P_VAR_NUMBER;
    DBMS_OUTPUT.PUT_LINE(V_RESULT_MSG);
END MY_TEST_PROCEDURE;
/
```

-- creamos el programa y establecemos sus dos argumentos

```
BEGIN
    DBMS_SCHEDULER.CREATE_PROGRAM(
        PROGRAM_NAME => 'MI_PROGRAM'
        ,PROGRAM_TYPE => 'STORED_PROCEDURE'
        ,PROGRAM_ACTION => 'MI_PROCEDURE'
        ,NUMBER_OF_ARGUMENTS => 2
        ,ENABLED => FALSE
        ,COMMENTS => 'PROGRAMA PARA MI PROCEDIMIENTO'
    );
END;
```

```

DBMS_SCHEDULER.DEFINE_PROGRAM_ARGUMENT(
  PROGRAM_NAME => 'MI_PROGRAM',
  argument_name  => 'P_VAR1',
  argument_position => 1,
  argument_type  => 'VARCHAR2',
  default_value  => ''
);

DBMS_SCHEDULER.DEFINE_PROGRAM_ARGUMENT(
  PROGRAM_NAME => 'MI_PROGRAM',
  argument_name  => 'P_VAR_NUMBER',
  argument_position => 2,
  argument_type  => 'NUMBER',
  default_value  => 0
);

dbms_scheduler.enable (name => 'MI_PROGRAM');
END;
/

-- creamos el job
BEGIN
  DBMS_SCHEDULER.CREATE_JOB (
    job_name => 'MI_JOB',
    PROGRAM_NAME => 'MI_PROGRAM'
  );

END;
/

-- probamos el código
DECLARE
  V_VAR1 VARCHAR2(50) := 'HOLA';
  V_VAR2 NUMBER := 2010;

BEGIN

  dbms_scheduler.set_job_argument_value(
    job_name      => 'MY_TEST_JOB',
    argument_position => 1,
    argument_value  => V_VAR1);

  dbms_scheduler.set_job_argument_value(
    job_name      => 'MY_TEST_JOB',
    argument_position => 2,
    argument_value  => V_VAR2);

  DBMS_SCHEDULER.RUN_JOB(
    JOB_NAME      => 'MY_TEST_JOB',
    USE_CURRENT_SESSION => TRUE);

END;
/

```

External script

Los jobs externos son aquellos que ejecutan scripts del sistema operativo. Para usarlos tenemos que definir el job como EXECUTABLE y crear un nuevo tipo de objeto llamado credential que contendrá el usuario y password del usuario del sistema operativo con el que queremos ejecutar el script.

Vamos a ver un ejemplo en el que haremos un expdp para hacer un export a las 11:30 todas las noches

1. Creamos el .par

```
cat expdp_tab.par
  userid=system/oracle@XEPDB1
  dumpfile=FULL_DB.dmp
  logfile=FULL_DB.log
  directory=backups
  full=y
```

2. Creamos una credencial para el usuario del S.O

```
BEGIN
  dbms_credential.create_credential (
    CREDENTIAL_NAME => 'ORACLEOSUSER',
    USERNAME => 'oracle',      -- usuario del S.O.
    PASSWORD => 'oracle@123',
    DATABASE_ROLE => NULL,
    WINDOWS_DOMAIN => NULL,
    COMMENTS => 'Oracle OS User',
    ENABLED => true
  );
END;
```

3. Creamos el job

```
Begin
Dbms_scheduler.create_job (
  job_name => 'BACKUP_FULLDB',
  job_type => 'EXTERNAL_SCRIPT',
  job_action=>'/oracle/app/oracle/product/12.1.0/dbhome_1/bin/expdp parfile=/export/home/oracle/expdp_tab.par',
  start_date => sysdate,
  Repeat_interval =>'FREQ=DAILY;BYHOUR=11; BYMINUTE=25',
  enabled => TRUE,
  credential_name=>'ORACLEOSUSER');
end;
/
```

Run_job

Para Ejecutar el job independientemente del scheduler podemos usar RUN_JOB al que se le pasa el nome del job y true/false según queramos que corra con sincronismo de la sesión en que está o asíncrona
Ejemplo:

```
BEGIN
    DBMS_SCHEDULER.RUN_JOB ('job_operacions');
END;
```

Monitorizar el status y log de un job

```
set pagesize 299
set lines 299
col JOB_NAME for a24
col actual_start_date for a56
col RUN_DURATION for a34
select job_name,status,actual_start_date,run_duration from DBA_SCHEDULER_JOB_RUN_DETAILS
where job_name='&JOB_NAME';
```

```
select * from DBA_SCHEDULER_JOB_LOG where job_name='&JOB_NAME';
```

Podemos ver la vista **all_scheduler_jobs**

```
select job_name,state,run_count,next_run_date from all_scheduler_jobs;
```

PRÁCTICA

Jobs plsql_block y stored_procedure

Haz un job que se ejecute todos los días que inserte en una tabla. Hazlo con un bloque plsql y con un procedimiento almacenado pasándole como parámetro el valor a insertar. Mira que funcione

PRÁCTICA

Jobs externos

Haz un job que se ejecute todos los días que haga un backup físico de la base de datos. Para ello tienes que usar rman pasándole un parámetro llamado cmdfile con el script que quieres que se ejecute y que contendrá el backup database